

# A Practical Guide to Flash attention

TOPIC -- FLASH ATTENTION

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

-- 01 --

**Un-embedding** An un-embedding layer is almost the reverse of an embedding layer. Whereas an embedding layer converts a token identifier into a vector, an un-embedding layer converts a vector into a probability distribution over tokens.

-- 02 --

The original version of the transformer architecture was proposed in the 2017 paper "Attention Is All You Need" by researchers at Google. The predecessors of transformers were developed as an improvement over previous architectures for machine translation, but have found many applications since. They are used in large-scale natural language processing, computer vision (vision transformers), reinforcement learning, audio, multimodal learning, robotics,, playing chess and at disaster responses. It has also led to the development of pre-trained systems, such as generative pre-trained transformers (GPTs) and BERT (bidirectional encoder representations from transformers).

-- 03 --

**Alternative activation functions** The original transformer uses ReLU activation function. Other activation functions were developed. The Llama series and PaLM used SwiGLU; both GPT-1 and BERT used GELU. Alternative activation functions are often used in combination with Gated Linear Units in the feedforward module.

-- 04 --

```
/* third sublayer */ z_d_copy ← copy(z_d) for each t in 1:length(z_d) do z_d[t] ←  
layer.layer_norm(z_d[t]) z_d ← layer.feedforward(z_d) for each t in 1:length(z_d)  
do z_d[t] ← z_d[t] + z_d_copy[t]
```

-- 05 --

One set of  $(W^Q, W^K, W^V)$  matrices is called an attention head, and each layer in a transformer model has

multiple attention heads. While each attention head attends to the tokens that are relevant to each token, multiple attention heads allow the model to do this for different definitions of "relevance". Specifically, the query and key projection matrices,  $W^Q$  and  $W^K$ , which are involved in the attention score computation, defines the "relevance". Meanwhile, the value projection matrix  $W^V$ , in combination with the part of the output projection matrix  $W^O$ , determines how the attended tokens influence what information is passed to subsequent layers and ultimately the output logits. In addition, the scope of attention, or the range of token relationships captured by each attention head, can expand as tokens pass through successive layers. This allows the model to capture more complex and long-range dependencies in deeper layers. Many transformer attention heads encode relevance relations that are meaningful to humans. For example, some attention heads can attend mostly to the next word, while others mainly attend from verbs to their direct objects. The computations for each attention head can be performed in parallel, which allows for fast processing. The outputs for the attention layer are concatenated to pass into the feedforward neural network layers. Concretely, let the multiple attention heads be indexed by  $i$ , then we have  $\text{MultiheadAttention}(Q, K, V) = \text{Concat}_{i \in [n \text{ heads}]} (\text{Attention}(X W_i^Q, X W_i^K, X W_i^V)) W^O$  where the matrix  $X$  is the concatenation of word embeddings, and the matrices  $W_i^Q, W_i^K, W_i^V$  are "projection matrices" owned by individual attention head  $i$ , and  $W^O$  is a final projection matrix owned by the whole multihead attention head. It is theoretically possible for each attention head to have a different head dimension  $d_{\text{head}}$ , but that is rarely the case in practice. As an example, in the smallest GPT-2 model, there are only self-attention mechanisms. It has the following dimensions:  $d_{\text{emb}} = 768, n_{\text{head}} = 12, d_{\text{head}} = 64$ . Since  $12 \times 64 = 768$ , its output projection matrix  $W^O \in \mathbb{R}^{(12 \times 64) \times 768}$  is a square matrix.

-- 06 --

In Multi-Token Prediction, a single forward pass creates a final embedding vector, which then is un-embedded into a token probability. However, that vector can then be further processed by another transformer block to predict the next

token, and so on for arbitrarily many steps into the future. This trades off accuracy for speed, since each new token costs just one more transformer block, rather than the entire stack.

-- 07 --

The attention mechanism used in the transformer architecture are scaled dot-product attention units. For each unit, the transformer model learns three weight matrices: the query weights  $W^Q$ , the key weights  $W^K$ , and the value weights  $W^V$ . The module takes three sequences, a query sequence, a key sequence, and a value sequence. The query sequence is a sequence of length  $\ell_{\text{seq, query}}$ , and each entry is a vector of dimension  $d_{\text{emb, query}}$ . Similarly for the key and value sequences. For each vector  $x_i$  in the query sequence, it is multiplied by a matrix  $W^Q$  to produce a query vector  $q_i = x_i W^Q$ . The matrix of all query vectors is the query matrix:  $Q = X_{\text{query}} W^Q$ . Similarly, we construct the key matrix  $K = X_{\text{key}} W^K$  and the value matrix  $V = X_{\text{value}} W^V$ . It is usually the case that all  $W^Q$ ,  $W^K$ ,  $W^V$  are square matrices, meaning  $d_{\text{emb, query}} = d_{\text{query}}$ , etc. Attention weights are calculated using the query and key vectors: the attention weight  $a_{ij}$  from token  $i$  to token  $j$  is the dot product between  $q_i$  and  $k_j$ . The attention weights are divided by the square root of the dimension of the key vectors,  $\sqrt{d_k}$ , which stabilizes gradients during training, and passed through a softmax which normalizes the weights. The fact that  $W^Q$  and  $W^K$  are different matrices allows attention to be non-symmetric: if token  $i$  attends to token  $j$  (i.e.  $q_i \cdot k_j$  is large), this does not necessarily mean that token  $j$  will attend to token  $i$  (i.e.  $q_j \cdot k_i$  could be small). The output of the attention unit for token  $i$  is the weighted sum of the value vectors of all tokens, weighted by  $a_{ij}$ , the attention from token  $i$  to each token. The attention calculation for all tokens can be expressed as one large matrix calculation using the softmax function, which is useful for training due to computational matrix operation optimizations that quickly compute matrix operations. The matrices  $Q$ ,  $K$  and  $V$  are defined as the matrices where the  $i$ th rows

are vectors  $q_i$ ,  $k_i$ , and  $v_i$  respectively. Then we can represent the attention as  $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$